UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 4

1. ENUNCIADO

Implemente dois programas paralelos em C com OpenMP: um limitado por memória, com somas simples em vetores, e outro limitado por CPU, com cálculos matemáticos intensivos. Paralelize com `#pragma omp parallel for ` e avalie o desempenho.

Analise quando o desempenho melhora, estabiliza ou piora, e reflita sobre como o multithreading de hardware pode ajudar em programas memory-bound, mas atrapalhar em programas compute-bound pela competição por recursos.

Explique quais são as melhores métricas para avaliar cada caso e como cada uma pode representar os diferentes aspectos do problema.

2. DISCUSSÃO

Ao longo do tempo, a capacidade de processamento do computador necessitou cada vez mais de velocidade para processar os problemas complexos demandados com precisão. Dessa maneira, a solução até então foi o aumento de clock dos processadores, que assim aumentava significativamente a velocidade de processamento. Contudo, isso chegou a um limite físico, como a geração excessiva de calor e o consumo alto de energia. Assim, fez-se a necessidade em pensar novas soluções para essa problemática, como a criação de processadores multicore, que possibilitou a programação paralela e melhora na programação concorrente.

- Programação Concorrente: "múltiplas tarefas progridem no mesmo período de tempo, mas não necessariamente ao mesmo instante", ou seja, o sistema operacional faz um escalonamento de processos através das políticas de escalonamento (como prioridade, Round Robin, etc.), necessitando salvar o

estado atual na PCB (Process Control Block), carregar o estado de outra tarefa, executar ela por um tempo, e depois repetir todo o processo de troca de contexto. Todavia, isso tem um custo, salvar e restaurar leva tempo.

* NOTA: precisa de no mínimo um núcleo para existir

- Programação Paralela: "múltiplas tarefas executam literalmente ao mesmo instante, em unidades de processamento diferentes", ou seja, cada núcleo tem sua própria unidade de processamento e registradores, trabalhando independentemente, e essas tarefas são executadas simultaneamente.

  * NOTA: precisa de múltiplas unidades de processamento.

No entanto, mesmo com essas inovações de hardware e processamento, os computadores ainda sofrem com o gargalo de rapidez na leitura da memória vs velocidade de processamento. Assim, segue os conceitos abaixo:

- Memory bound: quando um programa passa por memory bound, a CPU passa a maior parte do tempo ociosa esperando dados chegarem da memória, não calculando.
- Compute bound: a CPU está sempre ocupada calculando pois o processamento em si é pesado, mesmo que a memória consiga entregar dados rapidamente.

De maneira análoga, é como uma corrente com um elo mais fraco e outro mais forte. Quando a memória RAM é rápida, entrega muitos dados a CPU que não consegue suprir essa alta demanda (compute bound); quando a CPU é veloz, a memória não consegue suprir essa alta velocidade (memory bound). Além disso, deve-se considerar fatores físicos como o limite de velocidade de transferência, também que quanto mais velocidade, mais calor, o que necessita de um mecanismo de resfriamento otimizado, e mesmo o SSD NVMe rápido ainda é em ordens de magnitude mais lento que CPU.

A largura do cano, ou chamado de Bandwidth, é a quantidade de dados que pode ser transferida entre RAM e CPU por segundo.

- Bandwidth = quantidade de dados / tempo
- Unidade de medida: GB/s

Em relação ao bandwidth e quantidade de threads, um programa memory bound com uma thread, pode sobrar espaço disponível no bandwidth, já com múltiplas threads, pode saturar, criando filas sem real ganho.

- Tempo de transferência = tamanho dos dados / bandwidth

2.1. OpenMP

É uma API que permite escrever código paralelo em C de forma simples, ao invés de ter que manipular o loop a fim de paralelizar. Para compilar, tem que usar a flag -fopenmp

`gcc programa.c -o programa -fopenmp`

Por padrão, as variáveis elas podem ser shared ou private.

- Shared: as variáveis são declaradas fora do bloco paralelo, todas as threads leem e escrevem na mesma posição de memória;
- Private: cda thread tem sua própria cópia da variável, logo modificações de uma thread não afetam as outras.

2.2. Speedup E Eficiência

O Speedup é uma forma de analisar se o uso de paralelismo trouxe mais rapidez de processamento ou não. Assim, pode ser calculado pela relação:

- Speedup = Tempo com 1 thread / tempo com N threads
- Speedup > 1, paralelismo ajudou
- Speedup = 1, paralelismo não ajudou
- Speedup < 1, paralelismo piorou (overhead)
- Speedup = N, speedup linear, o que é ideal, mas raramente acontece
- Speedup > N, superlinear

A partir dessa relação, é possível calcular a Eficiência:

- Eficiência = Speedup / Nº de threads = T(1) / N x T(N)
- Eficiência = 100%, perfeito e sem desperdício (impossível na prática)
- Eficiência > 70%, bom aproveitamento
- Eficiência > 50%, aceitável
- Eficiência < 50, ruim (overhead dominando)

2.3. Lei De Amdahl

A lei de Amdahl explica que em infinitas threads, o programa não será infinitamente mais rápido, já que a parte serial do programa (inicialização, leitura de arquivos, print final, etc.) não pode ser paralelizada, isto é, não pode ser acelerada independe de quantas threads sejam usadas.

No memory bound, essa lei aparece como a parte serial sendo igual ao tempo esperando memória, porque não diminui com mais threads. No compute bound, o serial é a inicialização ou sincronização.

2.4. Etapas Para Resolução Da Tarefa

1º) Escreva o Programa 1 (memory-bound)

2º) Teste com 1 thread (garantir que está correto)
3º) Varie as threads e colete os tempos
4º) Calcule as métricas do Programa 1

5º) Escreva o Programa 2 (compute-bound)
6º) Teste com 1 thread (garantir que está correto)
7º) Varie as threads e colete os tempos
8º) Calcule as métricas do Programa 2
9º) Monte a análise comparativa
10º) Aplique a Lei de Amdahl
11º) Escreva as conclusões

2.5. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.6. OUTPUT

Segue no apêndice.

3. ANÁLISE DOS RESULTADOS

3.1. Programa 1

O Programa 1 realiza uma operação de soma elemento a elemento entre dois vetores de 100 milhões de elementos (vetor_soma[j] = vetor_x[j] + vetor_y[j]), exigindo 2 leituras e 1 escrita na memória RAM a cada iteração. Esse padrão caracteriza um programa memory bound, onde o gargalo é a banda de memória disponível, não a capacidade de cálculo da CPU.

Os resultados mostram três fases distintas:

- Fase 1 — Ganho expressivo (1 a 4 threads): O tempo caiu de 0.230s para 0.084s, uma redução de ~63%. Cada thread adicional trouxe ganho real e perceptível pois a banda de memória ainda não havia sido saturada.
- Fase 2 — Saturação (4 a 10 threads): O tempo oscilou entre 0.080s e 0.082s, sem redução significativa. A banda de memória foi completamente saturada por volta de 4 threads; adicionar mais threads não aumenta a velocidade de transferência de dados da RAM.

- Fase 3 — Overhead (acima de 8 threads): O speedup começa a cair levemente (3.12x com 8 threads, 3.09x com 10 threads), indicando que o custo de gerenciar mais threads supera o benefício.

Aplicando a Lei de Amdahl, considerando o speedup observado de ~3.16x, é possível estimar a fração paralela conforme disposto abaixo:

3.16 = 1 / (1 - P)

1 - P = 1 / 3.16

1 - P = 0.316

P = 0.684 -> ~68% do programa é paralelizável

Dessa forma, os ~32% restantes são limitados pela banda de memória, que seria a parte serial importa pelo hardware.

3.2. Programa 2

O Programa 2 realiza cálculos matemáticos intensivos (sin(j) * cos(j) + sqrt(j)) por iteração. Essas funções exigem muitos ciclos de clock da CPU internamente, caracterizando um programa compute-bound, onde o gargalo é a capacidade de processamento da CPU.

Os resultados mostram um comportamento completamente diferente do Programa 1, ao contrário dele, o speedup continuou crescendo em todas as faixas de threads testadas. Isso acontece porque cada núcleo adicional contribui com poder de cálculo real, sem depender da banda de memória.

Eficiência mais alta: A eficiência se manteve acima de 59% mesmo com 10 threads, contra apenas 31% do Programa 1 na mesma configuração. Isso indica que as threads estão sendo melhor aproveitadas.

Tendência de crescimento: Com mais núcleos disponíveis, o speedup provavelmente continuaria crescendo, pois o gargalo é computacional, e cada núcleo novo resolve esse gargalo diretamente.

Aplicando a Lei de Amdahl, considerando o speedup observado de ~5.88 x com 10 threads, é possível estimar a fração paralela conforme disposto abaixo:

5.88 = 1 / (1 - P)

1 - P = 1 / 5.88

1 - P = 0.170

P = 0.830 -> ~83% do programa é paralelizável

Dessa forma, os ~17% restantes representam overhead do OpenMP e partes sequenciais.

4. CONCLUSÃO

Os dois programas demonstram de forma clara como o tipo de gargalo de um programa impacta diretamente o ganho obtido com paralelismo.

O Programa 1 (memory-bound) saturou rapidamente a banda de memória com apenas 4 threads, tornando inútil o uso de threads adicionais. A eficiência despencou de 97% com 2 threads para apenas 31% com 10 threads, evidenciando desperdício de recursos. Para esse tipo de programa, mais threads não significa mais desempenho, já que o gargalo é o hardware de memória, não a CPU.

O Programa 2 (compute-bound) apresentou escalabilidade muito superior, com o speedup crescendo continuamente até 10 threads e eficiência mantida acima de 59%. Cada núcleo adicional contribuiu diretamente para reduzir o tempo de execução, pois o gargalo (os cálculos trigonométricos) é resolvido diretamente pela adição de mais poder computacional.

A Lei de Amdahl confirmou esses comportamentos: a fração paralelizável do Programa 1 (~68%) é menor que a do Programa 2 (~83%), explicando por que o compute-bound escala melhor.

Por fim, os resultados reforçam que identificar o gargalo do programa é essencial antes de paralelizar.

5. APÊNDICE

5.1. RESOLUÇÃO DO PROGRAMA 1 PARA CALCULAR A MÉDIA DAS REFERÊNCIAS

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h> // OpenMP
#include <time.h>

#define N 100000000 // pra alcançar a memória RAM e testar o memory bound
```

```c
int main() {
    // Criar os vetores
    double *vetor_soma;
    double *vetor_x;
    double *vetor_y;

    vetor_soma = (double *) malloc( (N)*sizeof(double) );
    vetor_x = (double *) malloc( (N)*sizeof(double) );
    vetor_y = (double *) malloc( (N)*sizeof(double) );

    if (vetor_soma == NULL || vetor_x == NULL || vetor_y == NULL) {
        printf("Erro: Faltou memória!\n");
        return 1;
    }

    // Programa 1: soma dos vetores

    // Atribuição de valores aos vetores
    for (int i=0; i<N; i++) {
        vetor_x[i] = (double) i;
        vetor_y[i] = (double) i;
    }

// Execução pra pegar o valor de referencia
    double tempo_inicio_p1_ref = omp_get_wtime();
    for (int j=0; j<N; j++) {
        vetor_soma[j] = vetor_x[j] + vetor_y[j];
    }
    double tempo_fim_p1_ref = omp_get_wtime();
    double delta_tempo_p1_ref = tempo_fim_p1_ref - tempo_inicio_p1_ref;

    printf("Tempo de referencia: %f seg\n", delta_tempo_p1_ref);

    free(vetor_soma);
    free(vetor_x);
    free(vetor_y);

    return 0;
}
```

5.2. RESOLUÇÃO DO PROGRAMA 1

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h> // OpenMP
#include <time.h>
```

```c
#define N 100000000 // pra alcançar a memória RAM e testar o memory bound
#define DELTA_TEMPO_P1_REF 0.253

int main() {
    // Criar os vetores
    double *vetor_soma;
    double *vetor_x;
    double *vetor_y;

    vetor_soma = (double *) malloc( (N)*sizeof(double) );
    vetor_x = (double *) malloc( (N)*sizeof(double) );
    vetor_y = (double *) malloc( (N)*sizeof(double) );

    if (vetor_soma == NULL || vetor_x == NULL || vetor_y == NULL) {
        printf("Erro: Faltou memória!\n");
        return 1;
    }

    // Programa 1: soma dos vetores

    // Atribuição de valores aos vetores
    for (int i=0; i<N; i++) {
        vetor_x[i] = (double) i;
        vetor_y[i] = (double) i;
    }

    // iniciar contagem de tempo
    double tempo_inicio_p1 = omp_get_wtime();

    // Soma dos vetores x e y
    #pragma omp parallel for // paralelizar
    for (int j=0; j<N; j++) {
        vetor_soma[j] = vetor_x[j] + vetor_y[j];
    }

    // Tempo final
    double tempo_fim_p1 = omp_get_wtime();
    double delta_tempo_p1 = tempo_fim_p1 - tempo_inicio_p1;
    printf("Threads: %d | Tempo: %fseg\n", omp_get_max_threads(),
delta_tempo_p1);

    // Cálculo do speedup e eficiecia
    double speedup_p1 = DELTA_TEMPO_P1_REF / delta_tempo_p1;
    double eficiencia_p1 = (speedup_p1 / omp_get_max_threads())*100;
    printf("Speedup: %.2fx | Eficiencia: %.2f%%\n", speedup_p1,
eficiencia_p1);
```

```
    free(vetor_soma);
    free(vetor_x);
    free(vetor_y);

    return 0;
}
```

5.3. RESOLUÇÃO DO PROGRAMA 2 PARA CALCULAR A MÉDIA DAS REFERÊNCIAS

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h> // OpenMP
#include <time.h>
#include <math.h>

#define N 100000000 // pra alcançar a memória RAM e testar o memory bound

int main() {
    // Criar as variaveis
    double resultado = 0.0;

    // Execução pra pegar o valor de referencia
    double tempo_inicio_p2_ref = omp_get_wtime();
    for (int j=0; j<N; j++) {
        resultado += sin(j) * cos(j) + sqrt(j);
    }
    double tempo_fim_p2_ref = omp_get_wtime();
    double delta_tempo_p2_ref = tempo_fim_p2_ref - tempo_inicio_p2_ref;

    printf("Tempo de referencia: %f seg\n", delta_tempo_p2_ref);


    return 0;
}
```

5.4. RESOLUÇÃO DO PROGRAMA 2

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h> // OpenMP
#include <time.h>
#include <math.h>

#define N 100000000 // pra alcançar a memória RAM e testar o memory bound
```

```c
#define DELTA_TEMPO_P2_REF 2.570

int main() {
    // Criar as variaveis
    double resultado = 0.0;

    // iniciar contagem de tempo
    double tempo_inicio_p2 = omp_get_wtime();

    // Cálculo pesado
    #pragma omp parallel for reduction(+:resultado) // para as várias
threads não ficarem tentando modificar resultados ao mesmo tempo
    for (int j=0; j<N; j++) {
        resultado += sin(j) * cos(j) + sqrt(j);
    }

    // Tempo final
    double tempo_fim_p2 = omp_get_wtime();
    double delta_tempo_p2 = tempo_fim_p2 - tempo_inicio_p2;
    printf("Threads: %d | Tempo: %fseg\n", omp_get_max_threads(),
delta_tempo_p2);

    // Cálculo do speedup e eficiecia
    double speedup_p2 = DELTA_TEMPO_P2_REF / delta_tempo_p2;
    double eficiencia_p2 = (speedup_p2 / omp_get_max_threads())*100;
    printf("Speedup: %.2fx | Eficiencia: %.2f%%\n", speedup_p2,
eficiencia_p2);


    return 0;
}
```

5.5. OUTPUT DOS VALORES DE REFERÊNCIA DO PROGRAMA 1 PARA O CÁLCULO DA MÉDIA

```
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> gcc -fopenmp tarefa4_media.c
-o tarefa4_media.exe
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_media.exe
Tempo de referencia: 0.258000 seg
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_media.exe
Tempo de referencia: 0.239000 seg
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_media.exe
Tempo de referencia: 0.266000 seg
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_media.exe
Tempo de referencia: 0.250000 seg
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4>
```

5.6. OUTPUT DO PROGRAMA 1

```
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=1
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4.exe
Threads: 1 | Tempo: 0.230000seg
Speedup: 1.10x | Eficiencia: 110.00%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=2
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4.exe
Threads: 2 | Tempo: 0.130000seg
Speedup: 1.95x | Eficiencia: 97.31%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=4
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4.exe
Threads: 4 | Tempo: 0.084000seg
Speedup: 3.01x | Eficiencia: 75.30%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=6
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4.exe
Threads: 6 | Tempo: 0.080000seg
Speedup: 3.16x | Eficiencia: 52.71%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=8
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4.exe
Threads: 8 | Tempo: 0.081000seg
Speedup: 3.12x | Eficiencia: 39.04%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=10
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4.exe
Threads: 10 | Tempo: 0.082000seg
Speedup: 3.09x | Eficiencia: 30.85%
```

5.7. OUTPUT DOS VALORES DE REFERÊNCIA DO PROGRAMA 2 PARA O CÁLCULO DA MÉDIA

```
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> gcc -fopenmp tarefa4_media2.c -o tare
fa4_media2.exe -lm
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_media2.exe
Tempo de referencia: 2.535000 seg
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_media2.exe
Tempo de referencia: 2.544000 seg
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_media2.exe
Tempo de referencia: 2.632000 seg
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_media2.exe
Tempo de referencia: 2.569000 seg
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4>
```

5.8. OUTPUT DO PROGRAMA 2

```
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> gcc -fopenmp tarefa4_p2.c -o tarefa4_
p2.exe -lm
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=1
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_p2.exe
Threads: 1 | Tempo: 2.540000seg
Speedup: 1.01x | Eficiencia: 101.18%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=2
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_p2.exe
Threads: 2 | Tempo: 1.445000seg
Speedup: 1.78x | Eficiencia: 88.93%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=4
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_p2.exe
Threads: 4 | Tempo: 0.747000seg
Speedup: 3.44x | Eficiencia: 86.01%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=6
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_p2.exe
Threads: 6 | Tempo: 0.518000seg
Speedup: 4.96x | Eficiencia: 82.69%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=8
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_p2.exe
Threads: 8 | Tempo: 0.456000seg
Speedup: 5.64x | Eficiencia: 70.45%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> $env:OMP_NUM_THREADS=10
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4> .\tarefa4_p2.exe
Threads: 10 | Tempo: 0.437000seg
Speedup: 5.88x | Eficiencia: 58.81%
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa4>
```